

Performance Evaluation of SDN-Enabled Switching System For IoT Infrastructure

Mohammad Naim Elham¹, Suriani Mohd Sam², Azizul Azizan³, Yusnaldi Md Yusof⁴, Norliza Mohamed⁵, Norulhusna Ahmad⁶
Razak Faculty of Technology and Informatics, Universiti Teknologi Malaysia
 Universiti Teknologi Malaysia,
 Kuala Lumpur, Malaysia

Abstract— The revolution of the Internet of Things (IoT) has had a major impact on the infrastructure of networks and information technology. Software Defined Networking (SDN) is a way to enhance infrastructure agility and thus facilitate dynamic and scalable design, delivery and operation of network services. It is therefore believed that the agility brought about by network programming or SDN is essential to address the IoT revolution which is pushing the infrastructure to its limit with numerous and diverse requirements to meet. IoT nodes usually depend on the network layers to communicate with each other. However, it is almost impossible to construct a single end-to-end infrastructure that addresses the whole set of IoT constraints. The objective of this project is thus to share components in a converged layer where Raspberry acts as an IoT gateway as well as SDN enabled device that can satisfy various IoT requirements. Raspberry Pi has been a common IoT commodity for quite a few years now and is targeted to be redesigned for additional networking functionality. The most recent version #4 of Raspberry Pi has been installed with a very well-known virtual Open vSwitch. The Raspberry Pi serves as the data path of network communication throughout the project with SDN controller Ryu operating on the Raspberry Pi to provide programmability. A system of wired, wireless and SDN networks combined is then designed to validate the operations of the Raspberry Pi. After successful functionality, the Raspberry Pi is then evaluated in terms of QoS parameters (bandwidth, packet loss, delay and jitter) to ensure the suitability of Raspberry Pi utilized as a common device for both IoT and SDN operations.

Keywords— *Internet of Things, Software Defined Network, Switching, Quality-of-Service.*

I. INTRODUCTION

The Internet of Things (IoT) allows the collection and exchange of information in real-time to provide heterogeneous services for billions of devices connected to the Internet. The explosive growth of the Internet of Things promises many exciting new opportunities. However, it also presents certain operational and performance issues and limitations. IoT will have a significant impact on the existing network infrastructure and will push the infrastructure to its limits, having to meet numerous and various requirements [1]. Therefore, IoT infrastructure demands new architectures and technologies that can support unpredictably increasing traffic in order to facilitate a massive number of connected devices with diverse and complex characteristics.

As network traffic is growing rapidly with the emergence of IoT communication, traffic forwarding becomes more costly. As reported by Business Insider, an American financial and business news website, on average for every IoT project 35% of the expenses are related to hardware as

reported in Business Insider, 2016. According to another report by Cisco, consumer and business hardware spending can reach approximately \$3 billion in 2020 [2]. As such, innovative approaches are needed and a cost-effective way of addressing the present challenges of IoT traffic management.

The Chinese telecoms firm Huawei estimates that the information and communications technology (ICT) industry could use 20% of the world's electricity and release more than 5% of the world's carbon emissions by 2025. In 2016, the US government's Lawrence Berkeley National Laboratory estimated that American data centres – facilities where computers store, process and share information – might need 73bn kWh of energy in 2020. The world is producing ever more electrical and electronic waste. The quantity of dumped computers, telephones, televisions and appliances doubled between 2009 and 2014, to 42 million tonnes per year globally and the trend is going to increase according to the statistics below [3].

To address these issues, software-defined networking (SDN) has emerged lately, to address the aforementioned IoT challenges and requirements in an attempt to decouple the network control plane from the data plane. The data plane consists of switches and routers that facilitate the exchange of packets in a network, whereas the 'control plane' interface serves as an interconnection between a controller and switches in the data plane and defines the interaction between them. SDN presents the applications in the top layer with an abstraction of the underlying network. It employs 'control plane' as a software element, which resides on a server, yet deploys 'data plane' on the network devices. Due to this flexibility in controlling the forwarding plane, allows network equipment to acquire new functionalities and must not be replaced when a new requirement is necessary. SDN has grown substantially in the industry over the last few years. The first vendors to provide OpenFlow features in their products included HP, NEC, and Pronto [4]; since then, the list has extended dramatically.

Although SDN has been a success, OpenFlow-compatible switches are still costly. You wouldn't think of an SDN-enabled switch investment of \$1,000 for an end-user home device [5]. As a result, flexible and dynamic SDN devices are still only deployed in companies with adequate budgets. Due to this constraint, there is a rising demand for low-cost SDN alternatives that could direct SDN to smaller-scale implementations such as home and low-budget enterprise environments. The idea of truly low-cost (i.e., devices costing less than \$100) hardware emerges to make SDN equipment accessible. Low-cost equipment is indeed not perfect, its low cost implies limitations and a problem.

II. SOFTWARE DEFINED NETWORK

Software Defined Networking (SDN) is the new approach for programmable networks. This approach transforms the process of network design, implementation and management. It has two distinctive features. First, it introduces a layer of abstraction that separates the control plane from the data plane. Separation of the control plane from the data plane makes network management considerably easier. In this way, computer software has an overview of the whole network, named as 'controller' and its whole responsibility is to be aware of the network state and undertake decision making. The hardware merely transmits the packets to its destination, as the controller has instructed. In general, instructions are a set of packet handling rules [6]. Secondly, a control plane consolidation establishes centralized management of networks where only a single program handles various different elements of a data plan (i.e. switches, routers ...) as opposed to the traditional paradigm where management is distributed and solitary devices have to be managed individually.

SDN traffic forwarding is based on simple 'match-action' rules in the data plane which significantly increases the decision making speed. According to a report from Google [7], SDN has helped increase their utilization of WAN at close to 100% utilization. The evolution of SDN has happened over 3 stages spanning almost 20 years of academic research and contribution [8]. In SDN architecture, controllers do not necessarily have to be single. Multiple controllers can run simultaneously to maintain a larger infrastructure size as a large enterprise or IoT network would need multiple controllers.

OpenFlow, which is standardized by the Open Networking Foundation (ONF), is the most common southbound interface. OpenFlow is a protocol describing the interaction with OpenFlow-compliant switches between one or more control servers. An OpenFlow controller installs flow table entries in switches so that according to these entries these switches can forward traffic. OpenFlow switches are therefore dependent on controller configuration. A flow is classified by match fields similar to ACLs and may contain wildcards.

The architecture of the SoftRouter also defines separate control and functionality of the data plane. It allows dynamic bindings between control elements and elements of the data plane, providing for a dynamic Internet Future. ForCES and SoftRouter are similar to OpenFlow and can play the southbound interface role. Also discussed are other networking technologies, as well as possible IETF southbound interfaces. For example, the Path Computation Element (PCE) and the Locator / ID Separation Protocol (LISP) are southbound interfaces candidates [9].

The upper layer comprises the application layer and the network applications which introduce unique network characteristics, enabling the control layer to manage flow tables and forwarding schemes [10]. This layer acquires from the controllers below the abstract and global view of the entire network, which in turn controls all networking devices in the infrastructure layer. The northbound interface is the interface between the application layer and the control layer as shown in Fig. 1.

The underlying layer includes the control layer and the control plane. This layer regulates all activity in the

underlying layer. In this situation, the controller plays a vital role between the application layer and the infrastructure layer [11]. When it comes to handling and programming switches, the Controller is considered to have all the brain. It interacts via the southbound interface with the switches situated in the lower layer i.e. infrastructure layer. The bottom layer comprises networking appliances, most of which are switches / routers that do not have the ability to take action within them and allow the controller to take over the flow tables. This is the flow table in a switch. This layer has the hardware that has implemented OpenFlow including the switches / routers [12].

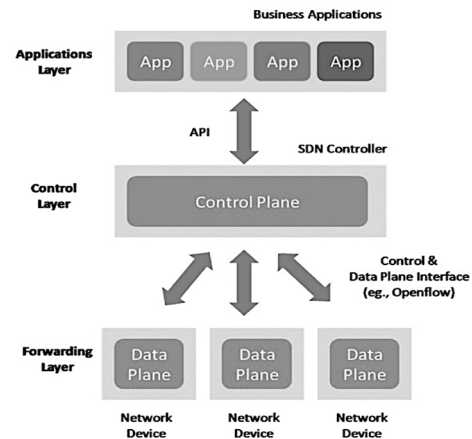


Fig. 1. SDN architecture

III. SYSTEM ARCHITECTURE

System architecture can often consist of system hardware and the integrated subsystems which will work with one another to perform the overall network. The system developed for this work consists of 3 sub-units. The wireless network unit, the wired network unit and mainly the SDN-enabled unit. Each of these units will be discussed furthermore in the coming sections. The controller will be positioned on a laptop for tracking purposes and convenience of configuration, and Ryu is just one of the controller software that has been chosen to use. Ideally, A software switch that can run on the Raspberry Pi, has functionalities that allow you to control the QoS parameters on the switch, as well as being easy to use will be utilized. If the controller is to be configured, it would be better suited if it were written in a programming language similar to python in order to reduce errors.

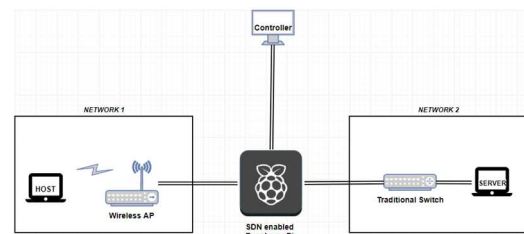


Fig. 2. System architecture

In Fig. 2, the architecture consists of two different types of network. The Network 2 consists of a typical LAN network where the host is connected to a traditional on the shelf switch. The connection is obviously wired and uses copper cable. Network 1 consists of an AP and a laptop acting as a server. The network resembles a typical WLAN network where the connection is through radio waves and Wi-Fi. The

SDN network is composed of Raspberry Pi which has Open vSwitch installed and configured on its bridge. The Raspberry Pi will be acting as a SDN enabled switch that can support a full-fledged SDN data plane device. However, for the control plane another laptop is directly connected to Raspberry Pi's original Ethernet port that can help Raspberry Pi in making decisions.

In terms of system hardware requirements, The wireless network unit is basically a WLAN network. It enables two or more devices to communicate using wireless radio waves to form a LAN within a limited area. As such, a LAN is a complete network in its nature where the members of the LAN can easily connect to each other without a secondary routing service. Any devices which can be accessed in a network of wireless medium are called stations (STA). The two types of STAs are wireless access point (WAP) and clients. The wireless network unit is responsible for representing a real world network. The project uses this network and specifically the host node to generate simulated traffic data with different parameters to pass through the network and reach the Network 2's server node. The wired network unit uses a single networking switch to create and manage all the connected devices in its own network as shown in Fig. 3. The switch serves as the essential connecting point and permits communication between devices like computers among each other. Basically it is a network of its own. However, it also needs to be able to communicate to other networks such as Network 1 where the host resides. A computer is appointed in the central point of this LAN. The computer acts as a server, listening to the Network 1's host node and responding to the packets.

The design of SDN switch for IoT communication necessitates the use of lightweight and low-cost components. Therefore, Raspberry Pi can be used as a low power, compact and economical single-board computer to execute the process of forwarding packets received from one port to another designated port in order for the packet to reach its desired destination. When combined with open source switching software (OpenFlow) [13], the end result is indeed a cost-effective IoT solution. The Raspberry Pi is operated by Raspbian operating system using the latest version (as of March 2020). The software kernel running the networking functionality is provided by Open vSwitch. This part of the network requires a controller to control the data paths of the Raspberry Pi. The Ryu controller is used to manage the vSwitch on a remote machine.

The system designed in terms of software requirement is quite diverse. It is essential for the software components to be well managed and for the whole system to work efficiently. As such, the next step to enable Raspberry as an OpenFlow switch is the necessity of adding a software switch to Raspberry Pi. The software switch used in this project is Open vSwitch. The schematics of how OpenFlow software switch is built on Raspberry is described in Fig. 4 below.

Raspberry Pi version 4 has 4 USB ports. Three of those 4 USB ports are equipped with USB-to-Ethernet dongles. By doing so, the extended number of Ethernet ports for Raspberry Pi from the original one port to a total four ports. The USB-to-Ethernet adapter works out of the box without having to set up additional drivers or apps on a Raspbian operating system. However, a ruling can be established in the file `/etc/udev/rules.d/70-persistent-net.rules` for a consistent mapping of interfaces to each of those of the device name.

However, for the ports to be able to communicate we need to create a network bridge among themselves. Otherwise, USB ports are not designed to communicate with each other by default but rather only to the RPi system. To create the bridge inside the Raspberry Pi, we will use the multi-layer open-source software switch namely Open vSwitch. Open vSwitch allows network developers to introduce a production quality switch platform on Linux-based systems. The features of OVS include standard management interface and provide programmatic expansion and transmission functions. Win32DiskImager is a simple Windows utility application that allows you to write images to the media (Write) or its acquisition (Read). The system is then capable of being prepared to see the SD card as bootable and to boot the operating system.

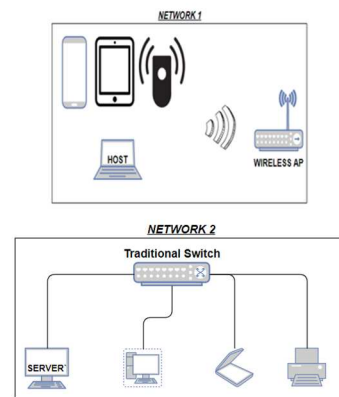


Fig. 3. Wireless network unit & wired network unit



Fig. 4. Software requirement for RPi

IV. SYSTEM CONFIGURATION

System configuration is a term in systems engineering that defines the computer hardware, the processes as well as the various devices that comprise the entire system and its boundaries. It also refers to the settings or the hardware-software arrangement and how each device and software or process interact with each other based on a system settings file created automatically by the system or defined by the user.

Access Points can be configured in terms of its SSID, channel, transmission power, MAC filtering, NAT functionality and many more. The only configuration that is crucial for this project is the IP assignment or the DHCP server. As mentioned in network architecture of the project, the wireless network has the 192.168.10.0 subnet. Thus, it is going to be distributing IP addresses from this pool.

A switch refers to a unit of the hardware that ties the network. By accommodating multiple devices, a switch links the entire Local Area Network (LAN). Since such a switch connects a range of network devices, the size can differ between 5 and 48 ports. Both these ports must handle cables that use the same network from various devices [19]. This managed network switch provides the configuration

allowance. As such, a static IP address has been assigned on one of its Fast Ethernet ports and also the hostname has been changed to reflect the project.

After making sure that the Raspberry Pi is updated to its latest distribution and installing Open vSwitch successfully as it was discussed previously, the next logical step is to configure Open vSwitch to operate as a virtual software switch based on Raspberry Pi and facilitate the data-path between the various USB-to-Ethernet connection amendments of Raspberry Pi. The very first `ovs-vsctl` command is the most essential to configure and control the virtual software switch. The idea is that to initialize the Open vSwitch generally provides a bridge between each interface of the device involved in the OpenFlow network. The actions of such a bridge would then be defined by the rules of flow. Firstly, the virtual bridge must be built and a virtual bridge interface with the same name will be generated automatically. This bridge can have more interfaces, if necessary. The `ovs-vsctl show` command prints a brief overview of the Open vSwitch database contents as shown in Fig. 5.

The OpenFlow switch can be connected to the controller through a TCP channel. It is also necessary to provide IP connectivity between the switch and the controller [14]. In this work the connection used is used through the bridge interface, which is also used for traffic routing, and thus assigned to the bridge. In this case, the IP address is not set to any network interface, but on the bridge, which is also found in the list of network interfaces, e.g. from the `ifconfig` command. The controller will be started from the directory with its source code using the `./bin/ryu-manager -verbose` command as shown in Fig. 6.

The OpenFlow switch can be connected to the controller through a TCP channel. It is also necessary to provide IP connectivity between the switch and the controller [14]. In this work the connection used is used through the bridge interface, which is also used for traffic routing, and thus assigned to the bridge. In this case, the IP address is not set to any network interface, but on the bridge, which is also found in the list of network interfaces, e.g. from the `ifconfig` command. The controller will be started from the directory with its source code using the `./bin/ryu-manager -verbose` command as shown in Fig. 7.

```
root@raspberrypi:/home/pi# ovs-vsctl show
2204c668-6879-4dc9-9bdf-ee5c929bc23a
Bridge "br0"
  Controller "tcp:192.168.0.100"
  fail_mode: secure
  Port "eth1"
    Interface "eth1"
  Port "eth0"
    Interface "eth0"
  Port "br0"
    Interface "br0"
    type: internal
  Port "eth2"
    Interface "eth2"
  ovs_version: "2.10.1" _
```

Fig. 5. OVS-vsctl show command

For the host and server configuration, the two nodes mentioned are 2 laptops booted from USB with Ubuntu Linux 18.04 LTS. The client node is configured to receive a dynamic IP address assigned by the wireless access point

DHCP server. This way it is going to receive one of the assigned IP addresses from 192.168.10.0 / 24 subnet pool. The server node is assigned a static IP address respectively

```
#!/bin/ryu-manager -verbose
loading app ryu.controller.ofp_handler
Instantiating app ryu.controller.ofp_handler of OFPHandler
BRICK ofp_event
loading app ryu.controller.ofp_handler
Instantiating app ryu.controller.ofp_handler of OFPHandler
BRICK ofp_event
CONSUMES EventOFPHello
CONSUMES EventOFPSwitchFeatures
CONSUMES EventOFPEchoRequest
CONSUMES EventOFPErrormsg
CONSUMES EventOFPPortStatus
CONSUMES EventOFPPortDescStatsReply
-----Continued in Appendix-----
```

Fig. 6. Ryu initialization process

The command for assigning IP address to a Linux machine is using the `ifconfig` command. The subnet used for this network is the default class C subnet mask 255.255.255.0 or the /24. The `up` command turns on the eth1 or whatever port selected as shown in Fig. 7.

```
root@projectpc-VirtualBox: /home/projectpc
root@projectpc-VirtualBox: /home/projectpc# ifconfig enp0s3 192.168.20.10 255.255.255.0
root@projectpc-VirtualBox: /home/projectpc# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 255.255.255.0 netmask 255.255.255.255 broadcast 0.0.0.0
    inet6 fe80::1e46:197d:2aa0:1f35 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:e1:71:5c txqueuelen 1000 (Ethernet)
    RX packets 2123 bytes 2014913 (2.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1036 bytes 98119 (98.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Fig. 7. Server IP configuration

V. EXPERIMENTAL SETUP

Although the technical details will vary with different tests, the experiment's overall design shall follow the common SDN architecture which is shown in Fig. 8. The lightweight and decentralized controller, Ryu, will run on that of the control plane. On the application layer, iPerf is going to be used to generate and monitor the impact of the QoS parameters. Two computers running the new Ubuntu Desktop 18.045 with 4.10 Linux Kernel will be used for our experimental setup. The test case utilizes Leaf-Spine topology in the network for benchmarking that has at least 1 SDN-enabled node. The SDN-enabled node should always be wired to the evaluation traffic generators HOST and traffic receiver SERVER.

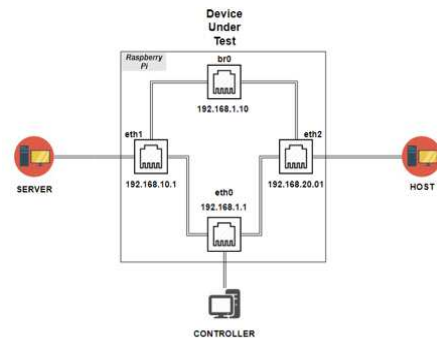


Fig. 8. Device under test diagram

The system developed in this project was brought into life with real hardware in my apartment hall. Three laptops were installed with the Ubuntu operating system to carry with Iperf and net-tools installed on them to carry out the validation and evaluation of this project. Two of the laptops served as Iperf client and server to emulate and calculate the generated traffic. The third laptop was directly connected to the

Raspberry Pi, was installed Ryu as SDN controller and was running as a simple learning switch to facilitate network switching services as shown in Fig. 9. A Wireless access point was set up with a DHCP server and connected wireless to one laptop and also wired to one of the Raspberry Pi's Ethernet ports. The managed traditional switch also was configured with a static IP address and connected to the second Ethernet Port of Raspberry Pi. The Raspberry Pi served as a physical medium to connect two wired networks with various IP subnets. Thus, it was deemed necessary to evaluate the data path that Raspberry Pi provides and as such, the evaluation phase of this project was performed to evaluate the efficiency of Raspberry Pi in a real world environment.

The packet stream between two network nodes is called a flow [15]. This stream must take the same route through a connection-oriented network, but the packet will take different paths within a connectionless network. It's likely that routes have different attributes with a connectionless network [16]. The four main network connections features are the throughput, packet loss, delay and jitter.

Throughput is a measure of the amount of data that can be sent over a link in a given amount of time. The throughput is measured in bits per second. The typical expression is in kilobits (10^3 bits), megabits (10^6 bits) or gigabits (10^9 bits) per second. The distinction between bandwidth and transmission is that measurements of the transmission may be influenced by large overheads which are not included in bandwidth calculations. The two factors affecting the throughput are, the amount of data transferred, and the time it took to transfer that data. Determining the throughput can be done in measuring the time it takes to transfer a predetermined amount of data. Iperf is a dynamic measurement tool with IP networks, where the primary aim is to assess the maximum reachable IP network throughput. It could also be used for other metrics such as jitter, and loss of packets.

Packet loss is defined as the percentage of packets that did arrive from source to recipient, to all sent packets in a certain unit of time. Packet loss can be evaluated either by ping command or Iperf UDP throughput evaluation. The ping is a command prompt that is used to test the computer's ability to reach a certain destination computer. The ping command works by sending the echo request message to the destination computer through the Internet Control Message Protocol (ICMP) [17]. The two main pieces of information provided by the ping command include how many such responses are returned and how long they will need to return.

The delay is the time it takes to send a packet or frame from a source node to a destination node. The delay is the product of three delays, these are called transmission delay, propagation delay and queuing delay. The delay is expressed in time, and since the delay usually is quite small, it is expressed in milliseconds. Round trip delays are the most useful for most applications because there is an interaction between the two communicating hosts. Ping and Iperf both can be used to evaluate the RTT of a network. Jitter is the variation in arrival times of successive packets from a source to a destination and is determined by the difference experienced by subsequent packets, RTTI and RTTI+1 [18]. All of the tools that can be used to measure RTT can also be used for inspecting Jitter including ping, trace route and Iperf. The tools specifically include Iperf when it is performed with UDP packets.

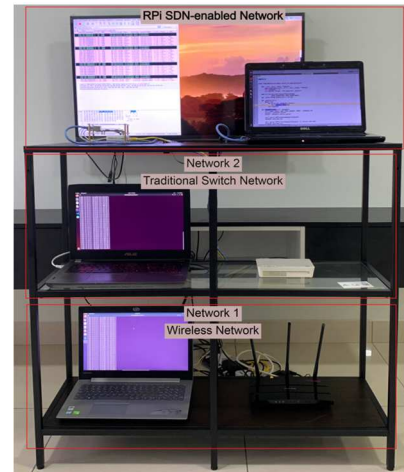


Fig. 9. Experimental setup in laboratory

The first stage collects the raw data from the network or computer. This can be done by active measurement, which are tools that generate traffic on the network to conduct the measurements. The Iperf program can output the results into text files for further inspection. The files are usually stored in directory C:\iperf3\text.txt after that program collects all information after a certain period of time.

VI. RESULTS AND ANALYSIS

Several test cases are implemented. In the first test case, the delay of the network link is to be measured and based on the measurement; the packet loss is to be determined. The delay could be evaluated as the amount of time it takes to send a packet from a host, until it did receive at the destination. But since this requires perfect synchronization of the clocks, an alternate method is mostly used. The latter calculates the delay in the context of the round-trip time. The maximum ICMP datagram depends on different OS types. On an Ethernet network, the maximum MTU is 1500. When a host transmits an IP datagram larger than the MTU size, the Raspberry Pi fragments the datagram into smaller chunks. This process can cause delay and packet loss. That is the reason why the test case considers to alternate packet size throughout the test to better reflect the performance ability of the Raspberry Pi in question.

The probability of packet drop usually increases with maximizing packet size. Given that more time is required to deliver a large packet from source to destination, the likelihood of congestion as well as other reasons for packet drops e.g. TCP retransmission timeout increases. Packet delay also rises with packet size increasing. A larger packet needs a bit more time to reach its destination compared to a smaller packet; not only, apparently due to more packet drops, more packet retransmission is needed; therefore, the delay generally increases. In IoT applications, the packet sizes are generally small such as health and wearable, smart assistance, home automation and work appliances. However, there are certain applications such as game consoles, smart cameras and smart TVs that require the ability of handling large packet sizes of data. The Fig. 10 shows the trend of how delay and packet loss are both tied to packet size according to the test case.

In the second test case, it measures the bi-directional and uni-directional available throughput of the connection. The

link between the two nodes shall be benchmarked and analysed by using the active measurement tool Iperf.

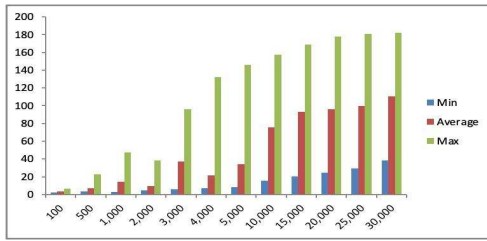


Fig. 10. Test Case #1 Trend

The test case will provide sufficient information to see network patterns, and determine whether the nodes manage to use the total available bandwidth. If the connection speed is not as anticipated, the bottleneck can be retrieved and removed. Whenever bidirectional throughputs are recorded, the total combined amount of data going within both directions is actually counted, and so bidirectional throughputs are usually simply twice as large as unidirectional, but in practice, whenever data is transmitted in both simultaneously, there is typically a performance decline in Raspberry Pi.

TABLE I. TEST CASE #2 RESULTS

No	Data Transfer (Mbytes)	Maximum Throughput (Mbits/s) (Client-Server)	Maximum Throughput (Mbits/s) (Server-Client)	Specified Theoretical Rate (Mbits/s)
1	112 Mb	93.5 Mb/s	-	100 Mb/s
2	113 Mb	-	94.9 Mb/s	100 Mb/s
3	118 Mb	98.8 Mb/s (Unidirectional)		100 Mb/s

The results are shown in Table I. The first test succeeded in achieving 93.5 Mb/s from Host to server and 94.9 Mb/s from server to client. The uni-directional test achieves 98.8 Mb/s which is very close to the theoretical maximum throughput. At the same time the CPU load was at maximum capacity. The theoretical throughput is supposed to be 100 Mb/s. It is not possible to get out of any connection with 100 percent utilization. Normally, 95% usage is about the limit that one can reach in the real world. If it gets more, the network will start saturating the connection and losing the packets.

The third test case measured the throughput available with a fixed amount of data and UDP packet type. Testing UDP performance in Iperf is different from testing TCP performance. Some parameter values such as throughput and then send data over a set period of time to observe the delay/jitter and also the percentage of lost datagrams. UDP tests can give us not only valuable information about jitter but also packet loss. Jitter is the variation in latency, and is not dependent on latency itself. High jitter can provoke serious problems and sometimes even break down VoIP calls. The UDP test is also able to measure the network's packet loss. A good quality link cannot afford to lose more than 1% of the total packets.

Table II shows the jitter with respect to the applied bandwidth. Although the jitter values are significantly low for nodes considered for the project it helps to visualize the congestion within the Raspberry Pi at high data rates. Notice the relatively high jitter values at high data rates for the nodes

in use. The jitter measured for this Raspberry Pi 4 model B is significantly low due to the fast processing of incoming packets. However, it is still limited by the 100 Mbits/sec Ethernet interface over the USB 2.0 shared hub.

TABLE II. TEST CASE #3 RESULTS

No	Fixed Throughput (Mbits/s)	Achieved Throughput (Mbits/s)	Jitter (ms)	Lost Packet / Total Packets	Packet Loss (%)
1.	10 Mb/s	10 Mb/s	1.68 ms	0/8504	0.00 %
2.	50 Mb/s	50 Mb/s	4.80 ms	25/42518	0.058 %
3.	100 Mb/s	95.8 Mb/s	13.56 ms	122/85034	0.143 %

The fourth test case is to measure the throughput split based on the number of clients. The Iperf is capable of emulating a number of clients up to 192. This test case is most relatable in situations in IoT where a vast number of devices try to connect and send data to a destination as part of their data collection process and the gateway device needs to fulfil the throughput requirements of all the connections simultaneously. The multiple thread parameters – P is client related and enables the client edge to concurrently run multiple threads. Of course, by using this parameter, the throughput will be split into the number of running threads. With this feature, we can simulate a number of nodes that might actively attempt to connect and send data to the server.

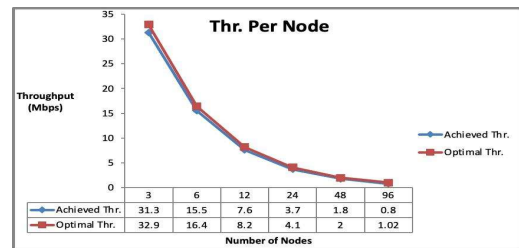


Fig. 11. Th. per node vs. number of nodes

As shown in Fig. 11 above, it can be explained that for testing throughput on 3 nodes who make requests simultaneously with the server, the client gets an average throughput of 31.3 Mb/sec. In addition we can also see that whenever fewer nodes are requesting to server, the larger the throughput gets disproportionately. Furthermore according to literature work, “parallel streams are able to achieve lower throughput by behaving as a single large stream that is the combination of n streams and gets an unfair share of the available bandwidth” [20]. The expected result is that the fair share of throughput is to be divided equally between each node; however, their work also predicted that there will be some degradation in performance due to many factors involved in the tests. Finally, it was concluded that the function of the RPi switch can run well but gets relatively unequal results when the number of nodes increases.

From Fig. 12, it can be explained that for testing the throughput of clients amounting to 3 to 96 clients, the results show that the total throughput has decreased, starting from 96.3 Mbps up to 2.21 Mbps. This can mean that whenever fewer nodes request data to the server, the total throughput increasingly meets the maximum bandwidth available by the system, which is around 98.8 Mbps. However, when many nodes make data requests to the server as shown in this test case, the decrease in throughput is caused by the bandwidth available by the system partly used for the communication

process between the server and the client. Because there are so many nodes communicating, eventually network traffic becomes even more congested. In this test the value of 96 is the maximum number of clients that can be handled by the system. It attempted to perform scenarios with 100 to 192 clients, but it resulted in error and the system was not able to send data. It can be concluded that 96 nodes are saturation points of the system.

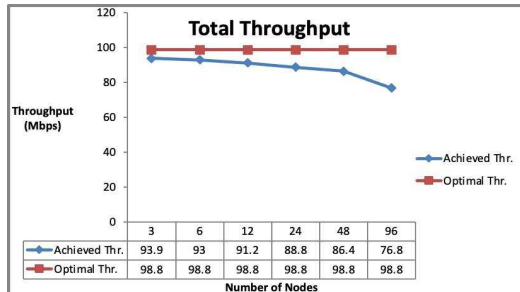


Fig. 12. Total throughput vs. number of nodes

The final test case to measure the packet loss ratio based on the number of clients for UDP packets. The Iperf is giving us the capability to emulate the number of clients as well as UDP packets by combining specific parameters. Packet loss aims to find out what percentages of packets were lost due to hardware or system limitations.

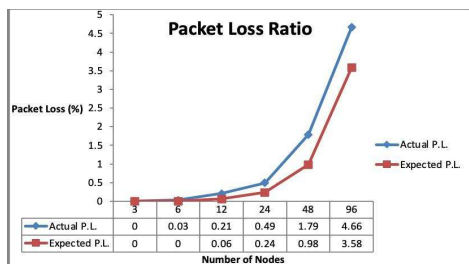


Fig. 13. Packet loss ratio vs. number of nodes

Illustrated in Fig. 13 above, it can be explained that for packet loss testing results have been obtained for accessing the number of Nodes starting from 3 to 96 nodes. In testing with 3 nodes that make requests to the server, the nodes get an average packet loss of 0%, which means that all data can be sent 100%.

VII. CONCLUSION

The first test case was performed using the already available ping command, which is a very common tool. The majority of the cases were performed by Iperf, an active measurement method, which is very common among network administrators to benchmark their networks. Each test case had different test criteria and different QoS parameters were evaluated. The test results were described in detail in the form of tables and charts. Tables were used for the accuracy of the results and charts were used for visualization purposes. As a result, the overall requirements of the Internet applications for QoS parameters were identified to validate the results of the Raspberry Pi in question. Software Defined Networking is a very interesting field of research with endless possibilities. Merging IoT with such a potent area of research can lead to significant innovations to very everyday things that are readily available. The current system only uses Open vSwitch

as a virtual switch. There are, however, many others available that could be appropriate for different scenarios. Same goes to the choice of the controller. In addition, with the capabilities of Software Defined Networking, the Raspberry Pi could be converted into a router, firewall or even a load balancing device, all depending on the programmability of the virtual switch used.

REFERENCES

- [1] Rechia, F. S., Syrotiuk, V. R., Ahn, G.-J., & Huang, D. (2016). An Evaluation of SDN Based Network Virtualization Techniques. Retrieved from https://repository.asu.edu/attachments/170581/content/StallRechia_asu_0010N_15909.pdf
- [2] Cisco. (2019). Defining the Future of the Internet About Cisco. Retrieved from www.cisco.com.
- [3] Wang, S., Chavez, K. G., Kandeepan, S., & Zanna, P. (2018). The smallest software defined network tested in the world: Performance and security. In IEEE/IFIP Network Operations and Management Symposium: Cognitive Management in a Cyber World, NOMS 2018.
- [4] Institute of Electrical and Electronics Engineers. (2016). 2015 IEEE Conference on Network Function Virtualization and Software Defined Network, NFV-SDN 2015. 2015 IEEE Conference on Network Function Virtualization and Software Defined Network, NFV-SDN 2015.
- [5] Chu, T. W., Shen, C. A., & Wu, C. W. (2018). The hardware and software co-design of a configurable QoS for video streaming based on OpenFlow protocol and NetFPGA platform. *Multimedia Tools and Applications*, 77(7), 9071-9091.
- [6] Huang, H., Zhu, J., & Zhang, L. (2014). An SDN-based management framework for IoT devices. In IET Conference Publications (Vol. 2014). <https://doi.org/10.1049/cp.2014.0680>
- [7] Y. Ma et al., "SDN test cases development and implementation," 2015 17th International Conference on Advanced Communication Technology (ICACT), Seoul, 2015, pp. 618-621.
- [8] Feamster, N., Rexford, J., & Zegura, E. (2014). The road to SDN: An intellectual history of programmable networks. In *Computer Communication Review* (Vol. 44).
- [9] T. Luo and S. Yu, "Control and Communication Mechanisms of a SoftRouter," COIN-NGNCON 2006 - The Joint International Conference on Optical Internet and Next Generation Network, Jeju, 2006, pp. 109-111.
- [10] Y. Jimenez, J. A. Cordero and C. Cervelló-Pastor, "Measuring robustness of SDN control layers," 2015 IFIP/IEEE International Symposium on Integrated Network Management (IM), Ottawa, ON, 2015, pp. 774-777.
- [11] Tayyaba, S. K., Shah, M. A., Khan, O. A., & Ahmed, A. W. (2017). Software defined network (SDN) based internet of things (IoT): A road ahead. *ACM International Conference Proceeding Series*, Part F130522. <https://doi.org/10.1145/3102304.3102319>
- [12] Yang, Y. Y., Yang, C. T., Chen, S. T., Cheng, W. H., & Jiang, F. C. (2015). Implementation of network traffic monitor system with SDN. *Proceedings - International Computer Software and Applications Conference*, 3, 631-634.
- [13] H. Kim, J. Kim and Y. Ko, "Developing a cost-effective OpenFlow testbed for small-scale Software Defined Networking," 16th International Conference on [14] C.-L. Tseng and F. J. Lin, "Extending scalability of IoT/M2M platforms with Fog computing," in 2018 IEEE 4th World Forum on Internet of Things (WF-IoT), 2018, pp. 825-830.
- [14] Ohira, Kenji. "Performance evaluation of an OpenFlow-based mirroring switch on a laptop/raspberry Pi." *Proceedings of The Ninth International Conference on Future Internet Technologies*. ACM, 2014.
- [15] C. Lin, T. Hu and H. Chan, "The implementation of multi-path delivery for data flows using Raspberry Pi boards in software-defined networks," 2017 IEEE 8th International Conference on Awareness Science and Technology (iCAST), Taichung, 2017, pp. 330-333.
- [16] M. Priyadarsini, P. Bera and R. Bampal, "Performance analysis of software defined network controller architecture—A simulation based survey," 2017 International Conference on Wireless Communications, Signal Processing and Networking (WISPNET), Chennai, 2017, pp. 1929-1935.
- [17] C. Fancy and M. Pushpalatha, "Performance evaluation of SDN controllers POX and floodlight in mininet emulation environment," 2017 International Conference on Intelligent Sustainable Systems (ICISS), Palladam, 2017, pp. 695-699.
- [18] R. G. Barba, M. Criollo, N. Aimacaña, C. Manosalvas and C. Silva-Cardenas, "QoS Policies to Improve Performance in Academic Campus and SDN Networks," 2018 IEEE 10th Latin-American Conference on Communications (LATINCOM), Guadalajara, 2018, pp. 1-6.
- [19] S. Han and S. Lee, "Implementing SDN and network-hypervisor based programmable network using Pi stack switch," 2015 International Conference on Information and Communication Technology Convergence (ICTC), Jeju, 2015, pp. 579-581.
- [20] A. Malishevskiy, D. Gurkan, L. Dane, R. Narisetty, S. Narayan and S. Bailey, "OpenFlow-Based Network Management with Visualization of Managed Elements," 2014 Third GENT Research and Educational Experiment Workshop, Atlanta, GA, 2014, pp. 73-74.